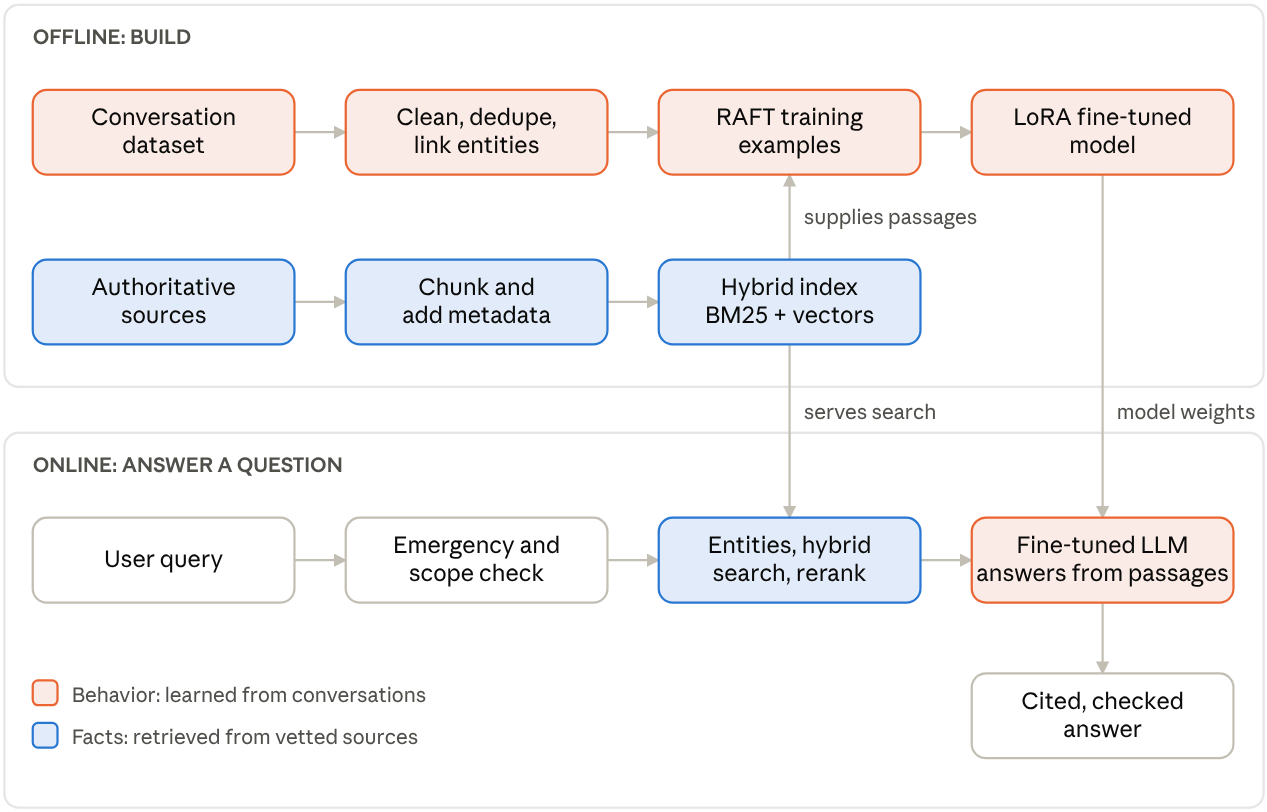
Medical Hybrid RAG + Fine-Tuning Roadmap

Use the conversation dataset to teach behavior and to build evaluations, retrieve every clinical fact from a separate authoritative corpus, and prove the hybrid earns its complexity with a four-arm ablation.

Behavior comes from conversations; facts come from a separate vetted corpus



end-to-end architecture · 2 offline pipelines, 1 online path

Two pipelines are built offline: conversations become training examples and a fine-tuned model, and vetted sources become a hybrid index. At question time the index feeds the model one passage set per query.

The key architectural point

The dataset (`integrated_conversations_ner_validated.csv`, 275,639 turns across 123,256 dialogues) is mostly patient questions and doctor replies of uneven quality. That makes it good for teaching a model *how to talk* and poor as a source of *facts*. If you retrieve from it, the model will present one anonymous reply as if it were clinical guidance.

So each asset gets one job, and one job only:

Asset	Use it for	Never use it for
Conversation dataset (MedDialog, HealthChat)	Tone, clarifying questions, answer structure, escalation behavior; realistic user queries for evaluation	Retrieved facts or citations
Authoritative corpus (MedlinePlus, PubMed / PMC, guidelines, drug labels)	Every clinical claim the model makes, with a citation	Teaching conversational style
NER entities (BC5CDR output)	Query understanding, metadata filters, slicing the data for analysis	Ground truth of clinical meaning; the labels are coarse
Held-out evaluation set	Measuring retrieval, grounding and safety	Training, prompt tuning or threshold tuning

Three consequences follow from this split:

- Fine-tuning changes *how* the model uses context, not *what it knows*. Updating a guideline means re-indexing, never retraining.
- Every answer can be traced to a source passage, which is what makes clinician review and audit possible.
- The conversation data needs less trust than the corpus, so the cleaning effort in step 2 matters most for what the model learns about style and safety, not facts.

1. Define scope and success criteria first

Scope decides your safety bar and which sources are acceptable, so fix it before any engineering. Answer these five questions in writing:

- **Audience:** patients, clinicians, or both. Patient-facing needs simpler language and stricter escalation.
- **Task:** general information, symptom triage, or both. Decide now that the system does not diagnose or prescribe.
- **Languages and regions:** guidelines, drug names and dosing differ by country, and the corpus must match.

- **Deployment:** a self-hosted open model or a hosted API. This sets your privacy options and whether you can fine-tune at all.
- **Regulatory exposure:** whether the output could count as clinical decision support where you operate. Check this with counsel early.

Then choose metrics. Each one below maps to a part of the system, so a failure tells you what to fix:

Metric	What it measures	How to measure it
Retrieval recall@k	Whether the right source passage is in the top k results	Hand-built question-to-source pairs
Faithfulness	Whether every claim in the answer is supported by the retrieved passages	LLM judge (RAGAS or similar), validated against human ratings
Citation accuracy	Whether each citation points to a passage that supports the claim	Automated check plus spot review
Escalation correctness	Whether red-flag cases go to urgent care and routine cases do not	Red-team set with a labelled expected action
Abstention quality	Whether the system declines when retrieval is weak instead of guessing	Questions the corpus cannot answer
Clarification behavior	Whether it asks before advising when key facts are missing	Deliberately under-specified queries

Set a minimum target for each metric with clinical input before you see any results. A target chosen after the fact only confirms what you built.

2. Fix the data issues before training

Clean the text first, then re-run NER, because entity counts depend on the text. Deduplicate before splitting, or near-identical turns will leak across train and test.

Issue	Evidence in the dataset	Fix	How to check
Template boilerplate	`chatdoctori` appears 1,399 times as a CHEMICAL; MedDialog replies carry sign-offs such as "Thanks for your query"	Strip greetings and sign-offs from `utterance`, drop the artefact entity	The term leaves the top-20 chemical list
Null utterances	139 turns in the file (the validation notes also say 198)	Drop them, recompute counts	Zero nulls remain
Near-duplicates	3,254 exact duplicates were removed, but paraphrased templates survive exact matching	MinHash with LSH on word shingles, or embedding similarity, with a threshold tuned on labelled pairs	Manually review a sample of flagged pairs
Entity variants	"Pain", "pain" and "pains" are counted separately	Lowercase and lemmatize, then map to canonical IDs	Unique-entity count drops
Weak NER labels	BC5CDR has only DISEASE and CHEMICAL, so "pain", "tooth", "throat", "smoking" and "alcohol" are mislabelled or weak	Link to UMLS or MeSH with scispaCy's entity linker, or add a symptom and anatomy model	Hand-label about 200 turns and compute precision per label
Inconsistent documentation	139 vs 198 nulls; noisy-entity share given as 1.77% and 24.20%; HealthChat described as simulated though it looks like real user-LLM chats	Recompute every figure from the file and correct the doc	Doc matches the recomputed numbers

Issue	Evidence in the dataset	Fix	How to check
Licensing and privacy	MedDialog, LMSYS-Chat and WildChat each have their own terms	Read each licence and scan for personal data	Written sign-off before commercial use

A starting point for the boilerplate strip. Tune the patterns against a sample of real turns, because an over-eager regex deletes clinical content:

```
import re

SIGNOFFS = [
    r"thanks? for (your )?query",
    r"hope i have answered your query",
    r"chat ?doctor\w*",
]
PATTERN = re.compile("|".join(SIGNOFFS), flags=re.IGNORECASE)

df = df.dropna(subset=["utterance"])
df["utterance_clean"] = df["utterance"].str.replace(PATTERN, "",
regex=True).str.strip()
```

After cleaning, re-run 04_scispacy_ner and then `05_ner_validation` again on `utterance_clean` and keep the old file as a versioned backup.

3. Build the RAG knowledge base

The knowledge base is a separate corpus of vetted medical sources, chosen to match your scope from step 1. Start with a few high-quality sources rather than many uneven ones.

Source	Contents	Best for	Check before use
MedlinePlus	Plain-language pages on conditions, tests and drugs	Patient-facing explanations	Reuse terms of each page

Source	Contents	Best for	Check before use
PubMed abstracts, PMC open-access subset	Research abstracts; full text for open-access papers	Evidence and detail for clinician use	Only the open-access subset allows bulk reuse; licences vary per article
Clinical guidelines (WHO, NICE, CDC, national bodies)	Recommendations and care pathways	Triage thresholds, standard of care	Region fit, licence, and the guideline's version
Drug labels (DailyMed, openFDA)	Official indications, contraindications, dosing and warnings	Drug and dosing questions	US-specific; match to your region

Chunking. Split by semantic section using the source's own headings, aiming for about 300-500 tokens with some overlap. Keep dosing tables and numbered procedures whole, and prepend the section path (for example "Asthma > Treatment") to each chunk so it stays interpretable on its own.

Metadata. Every chunk carries enough structure to filter, rank and cite it:

```
{
  "chunk_id": "medlineplus:asthma:treatment:003",
  "text": "...",
  "source": "MedlinePlus",
  "url": "https://...",
  "section_path": ["Asthma", "Treatment"],
  "published": "YYYY-MM-DD",
  "retrieved": "YYYY-MM-DD",
  "entities": [{"text": "asthma", "cui": "C0004096", "label": "DISEASE"}],
  "audience": "patient",
  "region": "US"
}
```

Freshness. Medical content goes stale. Store both the publication date and the retrieval date, schedule re-ingestion per source, show the date next to each citation, and prefer the newer or higher-authority passage when two sources disagree.

Shared entity vocabulary. Run the same entity extraction and UMLS linking over the chunks that you run over user queries. This is what lets the entity filters in step 4 match.

4. Build hybrid retrieval

Keyword search and dense search fail in opposite ways. BM25 finds exact terms such as drug names, doses and test names but misses paraphrase ("tummy ache" vs "abdominal pain"). Dense embeddings handle paraphrase but can blur exact terms. Running both and fusing the results covers each other's gaps.

Stage	What it does	Starting choice
Query understanding	Extracts entities, links them to UMLS concepts and expands synonyms	Your scispaCy pipeline plus its entity linker
Sparse retrieval	Ranks chunks by keyword match over text and entity names	BM25
Dense retrieval	Ranks chunks by embedding similarity to the query	BGE or E5; compare against a medical model such as MedCPT
Fusion	Merges the two ranked lists into one	Reciprocal rank fusion
Reranking	Rescores the merged candidates against the query with a cross-encoder	A cross-encoder reranker
Context assembly	Picks the top passages within the token budget, removes near-duplicates, keeps source and date	Rules in your own code

Reciprocal rank fusion needs no score calibration between retrievers. Each document's fused score is the sum, over the ranked lists R, of one divided by a smoothing constant k plus its rank in that list; k is commonly set to 60:

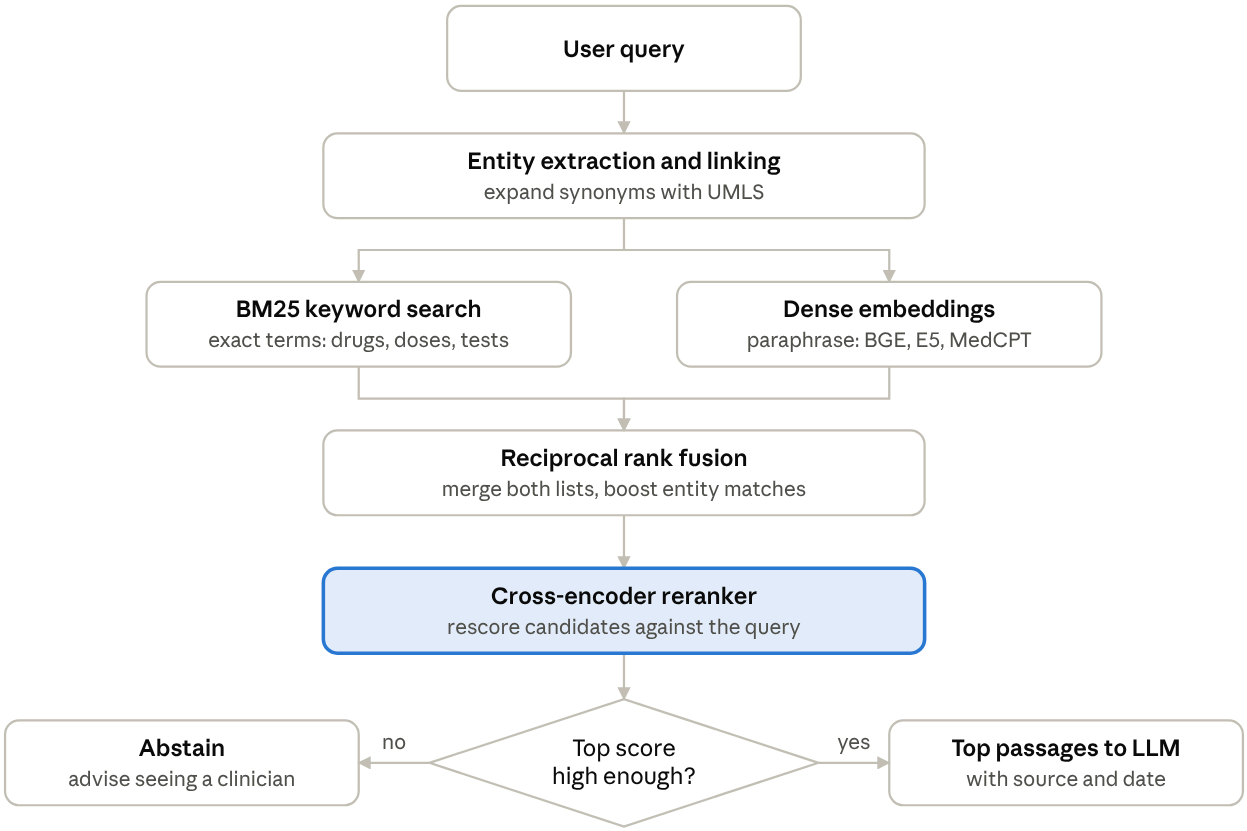
```
\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}
```

Use entities as boosts, not hard filters. NER misses things, and a hard filter on a missed entity hides the right passage. Boost chunks whose entities match the query's expanded entities, and hard-filter only on metadata you trust, such as region and audience.

Size the funnel by measurement. Take a wide candidate pool from each retriever, rerank a smaller set, and pass only a handful of passages to the model. Tune each number against recall@k and answer faithfulness on your evaluation set rather than by default.

Keep the reranker score. A low top score is the signal that the corpus cannot answer the question. The guardrail flow in step 8 uses it to make the model abstain instead of guessing.

Two retrievers feed one reranker, and its score decides whether to answer



hybrid retrieval pipeline · 6 stages, 1 decision

Both retrievers run on the entity-expanded query, fusion merges their lists, and the reranker's top score decides whether the model answers or abstains.

5. Get a RAG-only baseline working

Build the full retrieval-and-answer loop with an off-the-shelf instruct model and no fine-tuning. Everything that follows is measured against it, and you may find fine-tuning needs to do less than you expect.

1. Choose an instruct model (open or hosted) and fix a prompt that forces grounding.
2. Run it on your development questions and log the query, retrieved passages, answer and judge scores for every case.
3. Freeze the result as the baseline arm of the ablation in the roadmap.

A starting prompt:

```
You are a medical information assistant. Use ONLY the passages below.
- Cite passage ids like [P2] after each claim.
- If the passages do not answer the question, say so and advise seeing a
clinician.
- If key details are missing (age, duration, pregnancy, medications), ask one
clarifying question first.
- Never diagnose and never give a prescription dose.

Passages:
[P1] ...
[P2] ...

Question: ...
```

Then sort every failure by cause. This tells you whether to fix retrieval, prompting, training data or rules:

Failure	Likely cause	Where to fix it
The right passage never reaches the model	Retrieval	Step 4: chunking, embeddings, fusion, reranking
The right passage is present but the answer ignores or contradicts it	Generation	Prompt, then fine-tuning (step 6)
The answer reads well but cites nothing that supports it	Grounding	Fine-tune on grounded answers; stricter citation check
A red-flag case gets routine advice	Safety behavior	Fine-tuning plus the rule layer in step 8

Fine-tuning cannot repair bad retrieval. If most failures sit in the first row, return to step 4 before training anything.

6. Fine-tune with a clear purpose

Fine-tuning changes behavior, not knowledge. Use LoRA or QLoRA on an open model (Llama, Qwen or Mistral class) and train it to do five things, each with its own kind of example:

Behavior	Training example	Where the question comes from
Answer only from the passages and cite them	Question, passages, answer with `[P#]` citations	Cleaned MedDialog questions, answers rewritten against retrieved passages
Ask before advising	An under-specified question answered with one clarifying question	User turns missing age, duration, pregnancy or medications
Abstain when unsupported	A question whose supporting passage was removed, answered with a refusal and referral	The same questions with the oracle passage dropped
Escalate red flags	An emergency symptom answered with urgent-care advice	A curated, clinician-reviewed red-flag set
Keep a consistent tone and format	Every example	A short style guide applied during rewriting

Build the data RAFT-style: each example holds the question, the retrieved passages including some distractors, and a grounded answer. Distractors teach the model to ignore passages that do not apply. A training record can look like this:

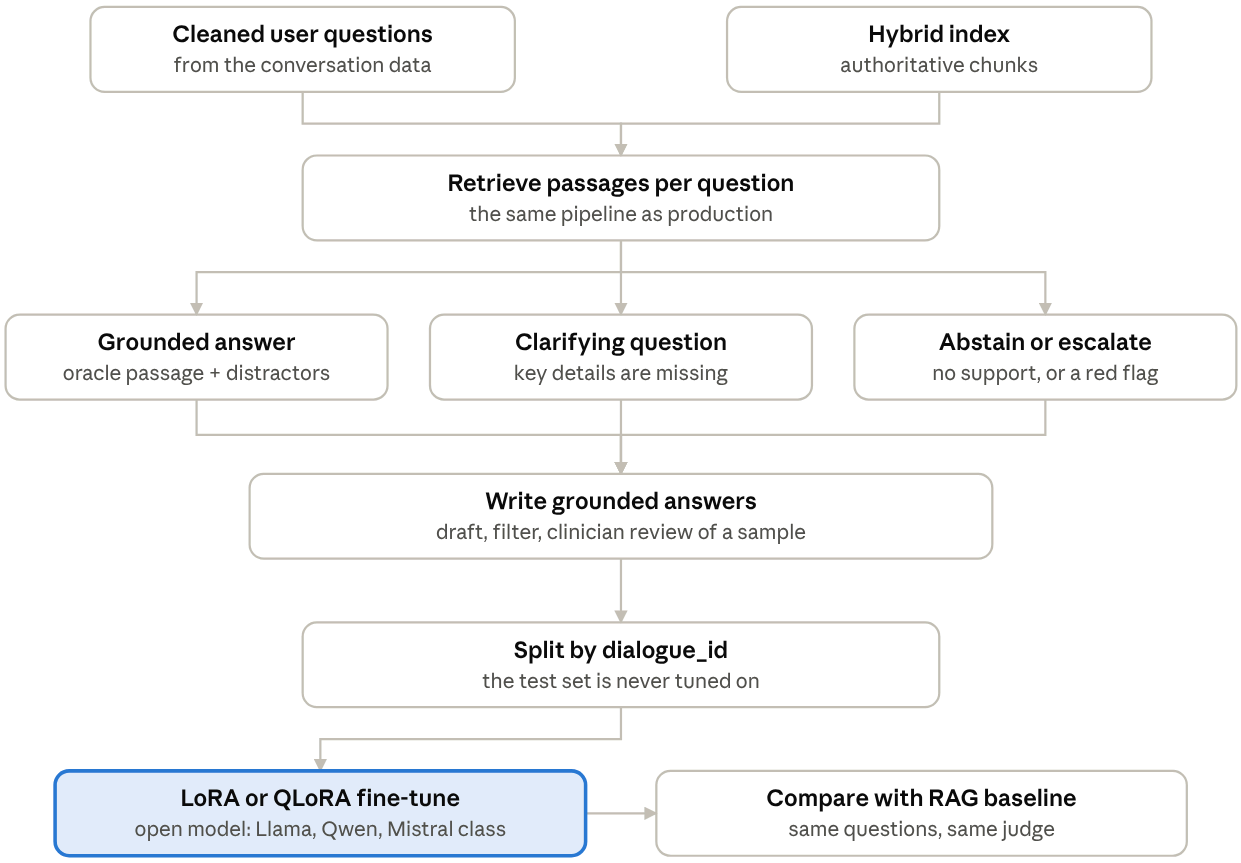
```
{"messages": [
  {"role": "system", "content": "Use only the passages. Cite them as [P#]."},
  {"role": "user", "content": "Passages:\n[P1] ...\n[P2] ...\n\nQuestion:
..."},
  {"role": "assistant", "content": "... [P1]"}
]}
```

Three rules keep the training honest:

- **Do not train on raw MedDialog replies.** They answer from memory and teach exactly the behavior you are trying to remove. Rewrite or regenerate each answer against its retrieved passages, then filter automatically and have a clinician review a sample.
- **Compute loss on the assistant turn only**, so the model learns the answers rather than the prompt format.

- **Split by `dialogue_id`, after deduplication,** and hold out a test set you never tune on. Start with a small LoRA rank and raise it only if the validation set shows underfitting.

Training examples teach three behaviors, and the test split is never tuned on



fine-tuning data and training flow · 3 example types

Every example is built from retrieved passages, so the model learns to use context in three ways: answer from it, ask for what is missing, or decline and escalate.

7. Evaluate each layer separately

A single end-to-end score hides where a failure happened. Measure each layer on its own data:

Layer	Question it answers	Data	Metric
Retrieval	Is the right passage retrieved?	Hand-built question-to-source pairs; even 200-300 helps a lot	recall@k, MRR

Layer	Question it answers	Data	Metric
Grounding	Is every claim supported by the passages?	Held-out test dialogues with their retrieved context	Faithfulness, citation accuracy (RAGAS or an LLM judge)
Knowledge sanity	Did fine-tuning damage general medical ability?	MedQA, PubMedQA, MedMCQA	Accuracy
Safety	Does it escalate, abstain and refuse correctly?	Red-team set: emergencies, dosing, self-harm, pregnancy, pediatrics	Correct-action rate
Human review	Would a clinician sign off?	A sample drawn from all layers above	Rubric score from clinicians

Validate any LLM judge against human ratings on a sample before you trust its scores.

The decisive experiment is a four-arm ablation on the same questions and the same judge:

Arm	Setup	What it tells you
A	Base model, no retrieval	The floor: what the model knows unaided
B	Base model + RAG	The baseline from step 5
C	Fine-tuned model, no retrieval	Whether tuning alone memorizes facts; expect weak grounding
D	Fine-tuned model + RAG	The hybrid; it must beat B on faithfulness and safety to justify the extra work

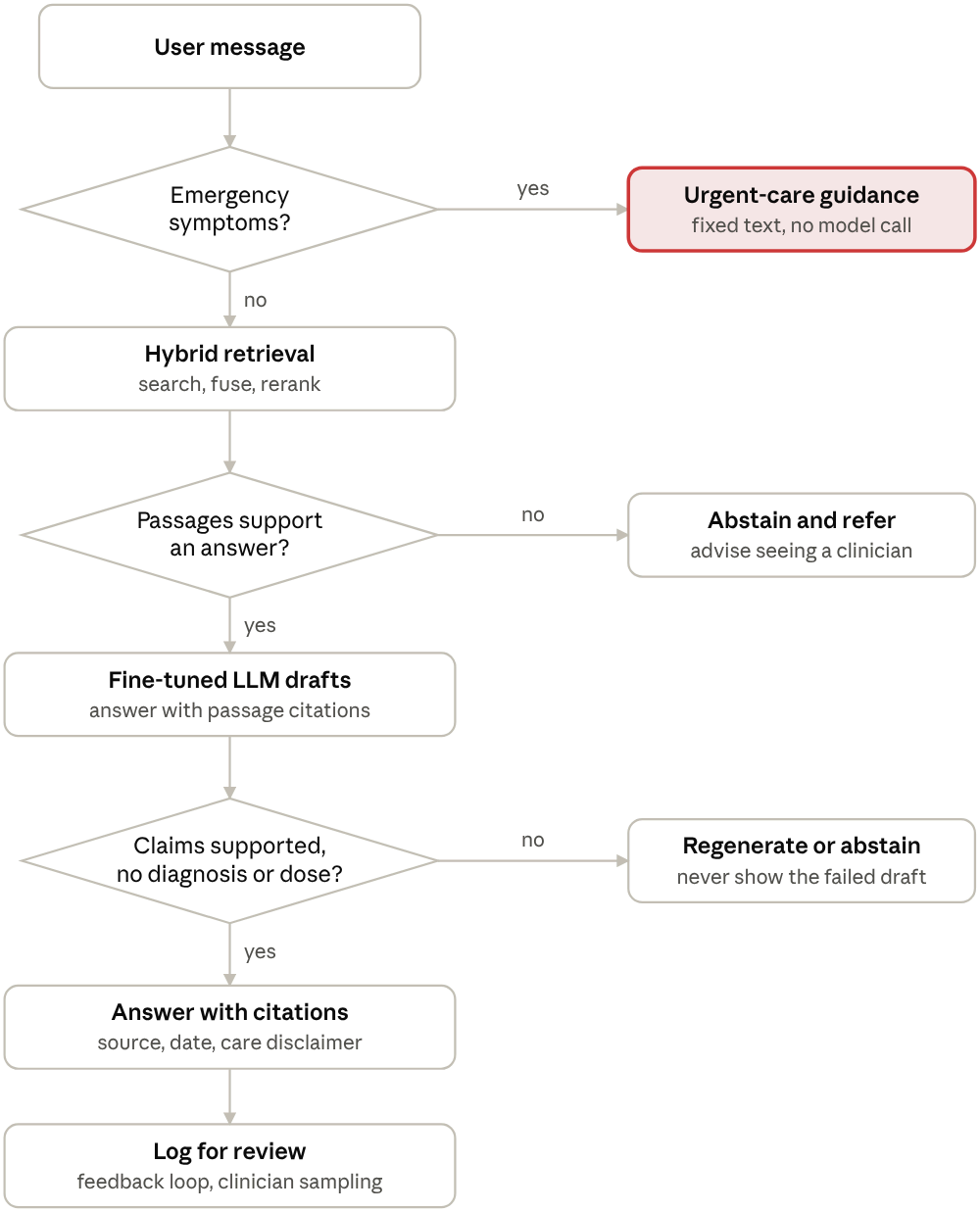
8. Add guardrails and deployment basics

Guardrails sit outside the model so they still work when the model is wrong. The flow below shows where each one runs.

- **Emergency detection before the model.** Rules plus a small classifier catch red-flag symptoms and return fixed urgent-care text. Clinicians should own the rule list.

- **Scope policy.** The system does not diagnose or give prescription doses. A dose may appear only if a retrieved label passage states it, and it is shown with that citation.
- **Citations and dates on every claim.** Show the source and its date so users and reviewers can check the answer.
- **Output check.** Verify that each claim is supported by a retrieved passage before display; regenerate once, then abstain.
- **Privacy.** Redact personal data before logging, set a retention period, and keep logs access-controlled.
- **Logging and feedback.** Record the query, passages, answer and each gate decision. Sample these regularly for clinician review and add every failure to the evaluation set.
- **Monitoring.** Track abstention rate, reranker scores and escalation rate over time. A shift usually means the corpus or the user population changed.

Three gates stand between a user message and a displayed answer



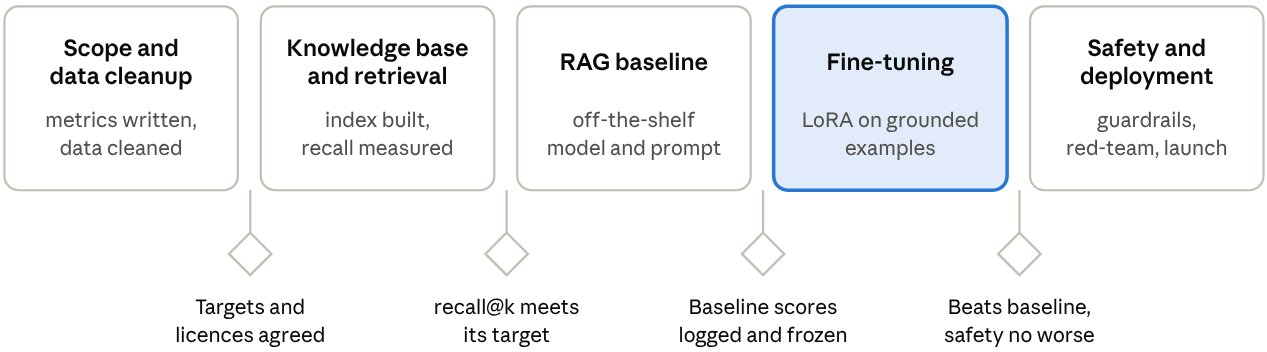
runtime guardrail flow · 3 gates, 3 exits

The emergency gate runs before any model call, the support gate uses the reranker score, and the output gate checks the draft before the user sees it.

Practical order of work

Work through the phases in order, and treat the four-arm ablation from step 7 as the last gate before launch. If arm D does not clearly beat arm B, ship the simpler RAG baseline and keep fine-tuning as a later option.

Fine-tuning comes fourth: the baseline and its gate come first



delivery roadmap · 5 phases, 4 gates

Each gate is a measured criterion from step 1, so a phase ends on evidence rather than on a date.

Risks

Risk	Why it matters	Mitigation
Stale or conflicting medical content	Guidance changes and sources disagree	Dated chunks, refresh schedule, prefer newer or higher-authority passages
Model answers from memory	Raw conversation replies teach it to	Rewrite answers against passages; check arm C in the ablation
Retrieval misses the right passage	The model then guesses	recall@k gate, abstain on a low reranker score, entity boosts
Leakage between data splits	Inflated scores hide real quality	Deduplicate first, then split by `dialogue_id`

Risk	Why it matters	Mitigation
Noisy entity labels mislead search	BC5CDR labels are coarse	Use entities as boosts, add UMLS linking, spot-check by hand
Licence or privacy breach	Three datasets with different terms	Review each licence, redact personal data, get sign-off
Users over-trust the answer	Fluent text sounds authoritative	Citations, disclaimers and the emergency gate

Open questions

- What is the intended use case?
- What is the compute budget (GPU type), and do you need a self-hosted open model or can you use an API? This mostly decides model choice and how aggressive the fine-tuning can be.